PHASE 1:

1A:

```
Updated 0 paths from the index
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

1B:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ find .pes/objects -type f
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/32/8baad8f00a1643772e776026398e590bfeae453814dfbc07ce7adf1351c584
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/41/97079c62ed3e7e82a32e99c61f70f007bae19b16f041b39650853baf342834
.pes/objects/13/51b6e4f16b3f6394dc321e5faa01ed12212e3dcb945d5fa1e9b80fc1ee4ef9
.pes/objects/3e/487782774d202155a16184922c934d5518eb6c29a8a3373ddc35289f19f43f
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

PHASE 2:

2A:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ nano tree.c
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test
_objects.o test_tree.o
rm -rf .pes
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

2B:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

PHASE 3:

3A:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ nano index.c
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ make pes
./pes status
cat .pes/index
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects
  untracked:  pes.c
  untracked:  Makefile
  untracked:  commit.h
  untracked:  pes.h
  untracked:  object.h
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  index.h
  untracked:  commit.c
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  object.c
  untracked:  mode
  untracked:  test_tree
  untracked:  test.txt
  untracked:  README.md

33188 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776445620 6 file1.txt
33188 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776445620 6 file2.txt
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

3B:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ cat .pes/index
33188 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776447759 1
2 hello.txt
33188 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 1776447759 8
 bye.txt
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

PHASE 4:

4A:

```
 1 file changed, 2 insertions(+)
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 328baad8f00a... Initial commit
Committed: 3e487782774d... Add world
Committed: 1351b6e4f16b... Add farewell
No commits yet.
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

4B:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/13/51b6e4f16b3f6394dc321e5faa01ed12212e3dcb945d5fa1e9b80fc1ee4ef9
.pes/objects/32/8baad8f00a1643772e776026398e590bfeae453814dfbc07ce7adf1351c584
.pes/objects/3e/487782774d202155a16184922c934d5518eb6c29a8a3373ddc35289f19f43f
.pes/objects/41/97079c62ed3e7e82a32e99c61f70f007bae19b16f041b39650853baf342834
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

4C:

```
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ cat .pes/refs/heads/main
1351b6e4f16b3f6394dc321e5faa01ed12212e3dcb945d5fa1e9b80fc1ee4ef9
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
arayana@Book4i5:~/PES2UG24CS079-pes-vcs$
```

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

ANS: Update .pes/HEAD to point to the new branch ref, then rebuild .pes/index to match the target branch's tree. On disk, write/overwrite files that differ and delete files that don't exist in the target branch. The complexity comes from handling deletions, dirty files, and the fact that a mid-crash leaves the repo in a broken half-switched state.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS: Hash the file on disk and compare it to the index — if they differ, the file is dirty. Then compare the index hash to the target branch's version of that file — if they differ, checkout would overwrite it. Refuse only if *both* are true. If both branches have the same version of the file, the dirty change survives checkout safely.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS: HEAD holds a raw commit hash instead of a branch ref, so new commits are created but no branch label moves forward — they're floating in the object store with nothing pointing to them. Switch away and those commits become unreachable, making them GC candidates. To recover, find the hash via reflog and create a branch pointing to it: git branch rescue a3f9c21.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS: Start from every branch tip and HEAD, walk every commit → tree → blob transitively, and add each hash to a hash set. Then scan all objects in the store and delete anything not in that set. For 100k commits with ~15 objects each, you'd visit roughly 200k–500k unique objects after deduplication from shared blobs across commits.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS: GC builds its reachable set, then a concurrent commit writes a new blob *after* the scan but *before* the sweep — GC deletes it, corrupting the repo. Git avoids this with a **2-week grace period**: it only prunes objects older than 14 days, so any freshly written object is safe even if GC hasn't seen it yet.